Testing Beyond Automation

Evidence, Doubt, and Operational Risk in the Age of Generative AI

Toward an Evidence-Centric, Observability-Driven Assurance Model for Complex Identity and Biometric Systems

Guillaume Harbonnier
Systems Engineering, QA Governance, Operational Assurance

Version 1.0 — Initial Proposal Draft — Not Peer Reviewed — May 16, 2026

Abstract

Generative AI is increasingly promoted as a way to automate software testing by producing test scripts, datasets, and scenario variants at scale. This paper argues that such productivity gains are real but incomplete. In complex socio-technical systems such as digital identity, biometric verification, border management, election ecosystems, and mission-critical governmental platforms, the strategic value of testing does not primarily reside in writing executable tests. It resides in the engineering act of doubt: validating hypotheses, confronting assumptions, discovering unknowns, challenging architectures, revising priors, and defining what evidence would be sufficient to trust an operational decision.

We propose an evidence-centric and observability-driven model of testing in which generative AI is used to industrialize controlled variability, not to replace test thinking. The engineer remains responsible for defining operational truth through truth matrices, precondition models, expected lifecycle states, expected artifacts, expected telemetry, and evidence boundaries. Generative AI may assist in building tools, scripts, validators, and generators that produce test data and scenario combinations, but these artifacts must remain auditable, reproducible, versioned, and governed under configuration management.

This version is released as an initial proposal draft, not as a peer-reviewed publication. Its purpose is to make the conceptual model explicit, invite critique, and provide a practical foundation for pilots in high-assurance quality engineering. It includes in-text citations, specifies ETSI TS 119 461 v2.1.1, introduces the STRAT-Q framing in the body, operationalizes Bayesian confidence and assumption exposure, clarifies the role of requirements as hypotheses, develops AI-assisted doubt under human governance, and provides practitioner workflow, tooling, metrics, case studies, and appendices.

The central thesis remains: AI can generate plausibility. Engineers remain accountable for operational truth.

Status: Version 1.0 initial proposal draft. This document has not undergone peer review. It should be read as a research and engineering proposal intended for critique, pilot implementation, and further validation.

Keywords: generative AI, test design, quality engineering, systems engineering, epistemic governance, observability, OpenTelemetry, digital identity, biometrics, operational assurance, risk compression, truth matrix, generated generators, context-driven testing, Bayesian confidence update.

Contents

1 Executive Summary

The public discourse around AI-assisted testing often starts with a seductive proposition: if generative AI can write tests, engineers can spend less time on testing and more time on strategy. This statement is partly true, but it hides a deeper question: what exactly is the strategic part of testing?

This paper argues that the strategic part is not merely choosing a framework, writing assertions, or increasing coverage. The strategic part is the cognitive, organizational, and operational process through which engineers confront uncertainty.

When an experienced engineer designs a test, the engineer often:

- validates hypotheses about system behavior;
- confronts assumptions embedded in requirements, architecture, data, and operations;
- discovers questions for which no answer is yet available;
- challenges architecture decisions and integration boundaries;
- consults product experts, domain experts, architects, operators, and business stakeholders;
- doubts initial interpretations and revises prior beliefs;
- defines which evidence would make a claim trustworthy.

This is the real value at risk when test design is fully outsourced to a statistical generation system.

Generative AI can industrialize variability. It should not silently replace strategic doubt.

The paper distinguishes two activities:

1. **Strategic test design:** a human-led engineering activity based on assumptions, hypotheses, priors, architecture challenge, operational context, and evidence design.
2. **Industrialization of test scenarios:** an automation-friendly activity in which scripts, generators, and AI-assisted tools expand scenario combinations, variable inputs, lifecycle states, expected artifacts, and telemetry checks at scale.

The strongest use of AI is not to directly invent opaque data. The stronger pattern is to ask AI to help build the generators, validators, scripts, and control mechanisms that produce and verify data under explicit governance. In this model, generator code becomes part of the test evidence. It can be reviewed, versioned, audited, and linked to requirements, design decisions, and system changes.

2 Problem Statement: The Risk of Automated Plausibility

Generative AI systems are very good at producing plausible continuations. They can generate code that looks correct, scenarios that look reasonable, and datasets that appear representative. However, plausibility is not validity. This distinction is consistent with AI risk-management guidance that emphasizes governance, measurement, and risk controls rather than unqualified trust in model outputs [2, 3, 25, 26].

Core risk: An AI system may generate a large number of plausible tests without questioning whether the operational model, assumptions, lifecycle states, data representativeness, observability expectations, or evidence boundaries are valid.

In high-criticality environments, such as identity and biometric systems, this risk is not theoretical. Digital identity assurance, biometric presentation attack detection, and remote identity proofing are governed by explicit requirements and evaluation practices in NIST, ISO/IEC, and ETSI references [1, 5, 6, 9, 11, 10]. A test may pass while the operational assurance claim remains false. For example:

- a biometric verification may return `SUCCESS`, while the audit chain is incomplete;
- an identity lifecycle transition may be accepted, while downstream systems remain inconsistent;
- an API may return an expected response, while the OpenTelemetry trace chain is broken;
- a border-control workflow may approve a transaction, while watchlist synchronization is stale;
- a test dataset may be syntactically valid, while it is not representative of operational populations, edge cases, or degraded modes.

The test appears to pass. The evidence is incomplete. The operational truth is not established.

3 Core Thesis

The central thesis of this paper builds on the long-standing recognition that test selection, test design, and testing psychology are not merely mechanical activities [12, 13, 14].

The central thesis of this paper is:

Testing in complex systems is not primarily a production activity. It is an epistemic and operational governance activity.

Generative AI has a legitimate and powerful role in testing, but its role should be carefully located. AI should be used to:

- expand controlled variability;
- build scenario generators;
- generate candidate test scripts for human review;
- create data-generation utilities;
- verify schema compliance;
- compare actual telemetry against expected telemetry;
- detect inconsistencies in evidence artifacts;
- assist in traceability and impact analysis.

But AI should not autonomously decide:

- what “true” means for the system;
- which assumptions are acceptable;
- which hypotheses matter most;
- which residual risks are tolerable;
- whether evidence is sufficient for a go/no-go decision;
- whether the architecture itself is coherent.

Those responsibilities remain engineering responsibilities.

3.1 STRAT-Q as a Conceptual Framing

This paper uses **STRAT-Q** as a compact label for the strategic quality and risk-compression framing developed throughout the argument. It is not introduced here as a commercial product or as a separate standard. In this paper, STRAT-Q simply denotes the governing idea that testing should connect strategic claims, assumptions, evidence, confidence updates, residual risk, and decisions.

The STRAT-Q framing can be summarized as follows:

$$Claim \rightarrow Assumptions \rightarrow Evidence \rightarrow Confidence\ Update \rightarrow Residual\ Risk \rightarrow Decision$$

This framing is used later when discussing possible dashboards and decision boards for residual-risk governance. A reader does not need any companion document to understand the paper: STRAT-Q is used only as a shorthand for evidence-centric, risk-compressing quality governance.

4 The Epistemic Nature of Test Design

This section formalizes the original intuition behind the paper: when an engineer writes a test, the engineer is often not simply implementing a procedure. The engineer is thinking with the system.

4.1 Writing a Test as Confrontation With the Unknown

A test is frequently born from a question:

> If the system is in this state, if these dependencies behave as expected, if this lifecycle transition is valid, if this data represents reality, and if this business rule is correctly implemented, then should the system produce this decision and this evidence?

This question contains multiple layers:

- explicit hypotheses;
- implicit assumptions;
- domain interpretations;
- architectural dependencies;
- operational constraints;
- uncertainty about data and timing;
- uncertainty about observability and evidence.

Good test design makes these layers visible.

4.2 Doubt as an Engineering Capability

Doubt is not hesitation. In engineering, doubt is a disciplined method for reducing operational risk.

The engineer asks:

- What am I assuming?
- Which assumption would invalidate the test?
- What evidence would falsify my expectation?
- Which component or expert should be consulted?
- What architecture decision is being implicitly trusted?

- What lifecycle state is required before execution?
- What would make this result unreliable?

A statistical generator may produce a test case. It does not automatically perform this interrogation unless the governance model forces it to.

4.3 Priors and Confidence Updates

Engineers enter a test design activity with priors: beliefs about how the system behaves, where defects are likely, which components are fragile, and which risks matter most. These priors may come from:

- architecture reviews;
- incident history;
- defect patterns;
- domain knowledge;
- operational constraints;
- expert judgment;
- business criticality.

Testing updates those priors.
A test is therefore not only a pass/fail mechanism. It is a confidence update mechanism.

5 From Assertions to Claims

Classical testing literature already shows that selecting meaningful test data requires more than executing arbitrary inputs [12, 13]. A classical test may be written as:

$$Input \rightarrow Expected\ Output$$

For complex systems, this is insufficient. We propose the following structure:

$$Claim \rightarrow Assumptions \rightarrow Evidence \rightarrow Confidence\ Update \rightarrow Residual\ Risk \rightarrow Decision$$

5.1 Definitions

Element	Meaning
Claim	A statement about the system that the test intends to support, such as "the biometric verification workflow rejects presentation attacks under defined conditions."
Assumptions	Conditions accepted for the test to be valid, such as environment version, dataset representativeness, service availability, policy configuration, sensor state, or observability availability.
Evidence	Observed artifacts, outputs, traces, logs, metrics, audit events, generated datasets, execution reports, configuration snapshots, and validation records.
Confidence Update	The change in belief produced by the evidence. It may be qualitative, scored, or probabilistic depending on governance maturity.

Residual Risk	The remaining operational exposure after the evidence has been interpreted.
Decision	A governance action, such as go, conditional go, no-go, additional testing, architecture review, remediation, or risk acceptance.

5.2 Assumptions, Hypotheses, and Requirements

A key distinction is required.

- **Assumptions** are contextual conditions accepted temporarily because the team needs them to reason or execute, but they are not yet directly validated. They must be documented, owned, risk-scored, reviewed, and revisited.
- **Hypotheses** are falsifiable propositions that require validation protocols, evidence plans, priors, and decision thresholds.

If a statement can be tested and falsified, it should be treated as a hypothesis. If it cannot be tested immediately but is required for execution, it becomes a governed assumption with an exposure score.

This distinction is especially important for QA and Business Analysis work. In practice, many requirements are written as if they were facts. In an evidence-centric model, a requirement should often be treated as a hypothesis about the system, the user, the operational environment, or the expected value of a behavior.

A requirements document is often a hypothesis register that has forgotten it is one.

Examples of hypotheses include:

- **Business requirement hypotheses:** the stated rule reflects the actual business or legal need.
- **User-workflow hypotheses:** operators or citizens will use the system according to the expected workflow.
- **Architecture hypotheses:** the proposed design can satisfy the required behavior under operational constraints.
- **Integration hypotheses:** external systems, queues, registries, or identity services behave within assumed contracts.
- **Lifecycle hypotheses:** object states and transitions are valid across the complete lifecycle.
- **Data hypotheses:** input data, test data, and reference data are representative enough for the claim being tested.
- **Observability hypotheses:** expected traces, logs, metrics, audit events, and artifacts are sufficient to reconstruct operational truth.
- **Risk hypotheses:** the residual risk after the evidence campaign remains below the decision threshold.

Many teams call all of these “assumptions.” This paper proposes a sharper operational rule:

1. Is the statement necessary for the decision?
2. Could the statement be false?
3. Can it be tested or falsified with evidence?

If the answer to all three questions is yes, treat the statement as a **hypothesis**. If it is necessary and could be false but cannot yet be tested, treat it as a **governed assumption** and compute its exposure.

5.3 A Minimal Bayesian Interpretation

The terms "prior" and "confidence update" should not be used as decoration. At minimum, they should express how evidence changes the team's confidence in a claim.

> **Example claim:** Remote biometric verification rejects presentation attacks under defined operational conditions.

A prior value such as

$$P(Claim\ Valid) = 0.70$$

must not be invented casually. It is an explicit prior belief before the new evidence campaign. It may come from:

- historical defect and incident data;
- previous campaign results;
- architecture maturity assessment;
- known weakness of components or integrations;
- observability readiness;
- expert elicitation;
- similarity with previous systems;
- risk-model outputs from a STRAT-Q or equivalent decision model.

In a mature implementation, the prior should be stored with provenance, for example:

```
"prior": {
  "claim_valid_probability": 0.70,
  "source": [
    "architecture_review",
    "previous_incidents",
    "expert_elicitation",
    "observability_readiness"
  ],
  "confidence_in_prior": "medium",
  "review_owner": "QA / Architecture"
}
```

The number 0.70 in this paper is illustrative. In practice, it could be derived in at least three ways:

1. **Historical estimation:** previous comparable campaigns show roughly 70% claim validity before remediation.
2. **Expert elicitation:** several experts estimate a plausible range, for example 0.60 to 0.80, with a central value near 0.70.
3. **Risk-model conversion:** an initial risk model estimates $P(Failure) = 0.30$, leading to $P(Claim\ Valid) = 1 - P(Failure) = 0.70$.

This is related to, but not identical to, the assumption exposure score R_0. The claim prior estimates belief in a testable proposition. The assumption exposure score prioritizes contextual conditions that may not yet be directly falsifiable.

A more Bayesian representation would avoid pretending that 0.70 is certain. For example, the prior belief may be represented as a beta distribution:

$$ClaimValidity \sim Beta(7, 3)$$

The mean is:

$$E[ClaimValidity] = \frac{7}{7+3} = 0.70$$

but the distribution also expresses uncertainty around that value.

A test campaign then runs 100 governed scenarios:

- 94 attacks are rejected correctly;
- 4 attacks are rejected functionally, but telemetry evidence is incomplete;
- 2 attacks are accepted incorrectly.

A naive functional interpretation would emphasize the 98% functional rejection rate. An evidence-centric interpretation separates three dimensions:

$$Functional\ Success = \frac{94+4}{100} = 0.98$$

$$Evidence\ Completeness = \frac{96}{100} = 0.96$$

$$Critical\ Failure\ Rate = \frac{2}{100} = 0.02$$

A simple beta-binomial functional update, using the 98 functional successes and 2 failures, would produce:

$$Posterior \sim Beta(7+98, 3+2) = Beta(105, 5)$$

with posterior mean:

$$E[Posterior] = \frac{105}{110} \approx 0.955$$

However, this posterior should not be interpreted as full operational assurance because evidence completeness and critical-failure severity must also be considered. Therefore, an evidence-centric decision model combines multiple posterior or scored dimensions:

$$DecisionConfidence = f(FunctionalPosterior, EvidenceCompleteness, CriticalFailurePenalty, Assumptior$$

A simplified operational scoring model could be:

$$Confidence_{decision} = w_f \cdot FunctionalPosterior + w_e \cdot EvidenceCompleteness - w_c \cdot CriticalFailureRate - w_a \cdot Ass$$

where w_f, w_e, w_c, w_a are governance-defined weights reflecting system criticality.

For high-assurance biometric systems, critical failures may receive a much higher penalty than normal assertion failures:

$$w_c \gg w_f$$

A possible decision could be:

> Conditional GO is not granted until the two false accepts are analyzed and missing telemetry paths are corrected. Functional behavior is promising, but evidence completeness, critical failure exposure, and residual risk are not yet acceptable.

This example is deliberately simple. The point is not to claim complete Bayesian rigor. The point is to make explicit where a prior comes from, how evidence changes it, and how residual risk remains governed.

5.4 Assumption Exposure Score

For assumptions that cannot immediately be falsified, the following exposure score can help prioritize governance:

$$R_0 = P(Assumption\ False) \times Impact \times DependencyMultiplier$$

R_0 is not a probability. It is a dimensionless exposure score used to prioritize which assumptions deserve review, evidence collection, or conversion into falsifiable hypotheses.

This creates a practical distinction:

- **Hypothesis:** assign a prior, define falsification conditions, collect evidence, and update posterior confidence.
- **Assumption:** estimate exposure R_0, assign an owner, define a review trigger, and decide whether it must be converted into a hypothesis.

In other words, assumptions are not left as informal notes. They become governed risk objects.

5.5 Scaling the Bayesian View Beyond a Single Claim

The simple scoring model above can be extended toward Bayesian networks when claims and assumptions are dependent. For example, the claim "presentation attacks are rejected" may depend on assumptions about the Presentation Attack Detection (PAD) service, sensor quality, policy configuration, watchlist freshness, and telemetry availability.

A lightweight dependency model can express that:

$$P(Claim) = f(A_{PAD}, A_{Sensor}, A_{Policy}, A_{Telemetry}, E)$$

where E represents observed evidence. The objective is not to over-mathematize the process. The objective is to prevent evidence interpretation from being reduced to a binary test result when the real assurance claim depends on multiple uncertain conditions.

In practice, teams can calibrate weights according to criticality. In a low-risk internal workflow, functional correctness may dominate. In a biometric or border-control workflow, critical-failure penalties and evidence-completeness penalties should dominate because a false accept or missing audit chain may create disproportionate operational exposure.

A possible critical-system weighting may be:

$$w_c = 10 w_f$$

where w_c is the critical-failure penalty and w_f is the functional-success weight. This is not a universal formula. It is a governance choice that should be explicitly documented and reviewed.

6 The Real Test Model for Complex Systems

In tightly coupled or complex interactive systems, local correctness is often insufficient to infer system-level safety or reliability [16, 20]. A test in a mission-critical system is never only:

$$Input \rightarrow Expected\ Output$$

It is closer to:

$$
\begin{aligned}
Test = {} & System\ State \\
& + Preconditions \\
& + Dependencies \\
& + Operational\ Context \\
& + Timing \\
& + Observability \\
& + Expected\ Artifacts \\
& + Evidence\ Trustworthiness
\end{aligned}
$$

6.1 Operational Context as a Validation Object

Operational context includes:

- system configuration;
- lifecycle state;
- data state;
- identity status;
- biometric capture conditions;
- external dependency status;
- queue and message-broker conditions;
- certificate validity;
- feature flags;
- network latency;
- operator workflow;
- observability availability.

If the context is invalid, the result is not meaningful. This is also why architecture description and test documentation practices should make stakeholder concerns, viewpoints, test bases, test items, and evidence expectations explicit [8, 7].

6.2 Pre-Requisite Verification

Before executing a test, the framework must verify:

- the environment is coherent;
- the system is in the expected lifecycle state;
- dependencies are reachable;
- datasets are valid and representative;
- contracts, APIs, queues, certificates, and configurations are aligned;
- observability pipelines are active;
- produced evidence can be trusted.

An invalid precondition should not be reported as a product defect. It should be reported as an invalid test context.

7 Complex Identity and Biometric Systems

This paper intentionally moves beyond simple API testing. The domain framing is informed by digital identity assurance guidance, biometric evaluation practices, privacy/legal risk references, and identity-proofing requirements [1, 11, 10, 5, 6, 9]. The target domain is complex socio-technical systems such as:

- national identity systems;
- civil registry ecosystems;
- biometric enrollment and verification systems;
- border-management platforms;
- election identity verification processes;
- citizen-service platforms;
- multi-agency operational workflows.

These systems are not merely software applications. They are operational trust infrastructures.

7.1 System-of-Systems Characteristics

Identity and biometric ecosystems typically involve:

- citizens or travelers;
- operators and supervisors;
- capture devices;
- biometric matchers;
- identity registries;
- policy engines;
- audit systems;
- external watchlists;
- certificate authorities;
- inter-agency integrations;
- operational dashboards.

A local output may be correct while the global assurance claim remains invalid.

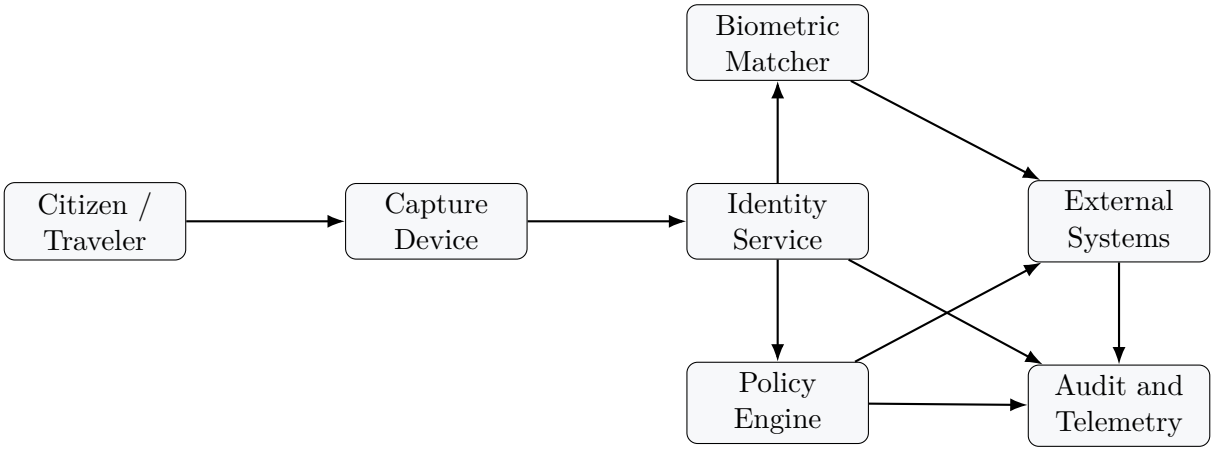

Figure 1: Complex identity and biometric validation as a system-of-systems problem.

8 Parametrized Testing and the Truth Matrix

Once strategic reasoning is complete, automation becomes powerful. At that point, engineers have defined:

- hypotheses to validate;
- assumptions to challenge;
- lifecycle states to cover;
- object states and transitions;
- input variability;
- expected outputs;
- expected artifacts;
- expected telemetry;
- preconditions;
- evidence rules.

Generative AI can then help expand combinations.

8.1 Truth Matrix Definition

A truth matrix is a governed representation of expected operational truth. It is compatible with model-based testing and property-based testing traditions, but extends them with telemetry, evidence records, and operational-risk decisions [24, 23, 7].

It maps:

- scenario context;
- citizen or object state;
- identity lifecycle state;
- biometric condition;
- input variables;
- expected decision;
- expected outputs;
- expected telemetry;
- expected artifacts;
- evidence acceptance criteria.

Citizen State	Identity State	Biometric Context	Operational Context	Decision	Expected Evidence
New applicant	Not enrolled	Good quality capture	Normal enrollment	Enroll	Enrollment event, identity record, audit trace
Active traveler	Active	Good match score	Border crossing	Approve	Verification span, policy span, audit event

Watchlisted person	Active	Match score above threshold	High risk context	Escalate	Alert event, supervisor workflow, full trace chain
Expired credential	Inactive	Any biometric input	Normal context	Deny	Denial event, lifecycle reason, audit record
Presentation attack	Active	Spoof attempt	Remote verification	Reject	PAD alert, rejection decision, evidence package

The AI may explore combinations. It may suggest additional rows. It may generate candidate cases. But the meaning of "true" remains defined by engineers.

9 Telemetry as Oracle

In traditional testing, the oracle is often an expected output. In complex operational systems, the oracle must also include telemetry.

Telemetry is not merely a debugging aid. In evidence-centric testing, telemetry is part of the oracle.

OpenTelemetry (OTel) is used here as a representative standard for traces, metrics, logs, context propagation, and semantic instrumentation [4]. The broader idea is independent of any single tooling stack: observability artifacts can become evidence artifacts.

9.1 Expected Observability Signals

A test scenario should specify expected:

- traces;
- span names;
- span IDs and trace IDs;
- parent-child relationships;
- business events;
- logs;
- metrics;
- audit artifacts;
- evidence packages.

Example: a biometric verification scenario may require:

- one capture span;
- one biometric-quality-evaluation span;
- one matcher span;
- one policy-decision span;
- one audit-write span;
- consistent trace ID propagation;

- required attributes such as transaction ID, identity lifecycle state, and decision reason.

If the output is correct but the telemetry chain is broken, the evidence is incomplete.

9.2 Same Trace ID and Evidence Continuity

A critical requirement is trace continuity. When a test runs, the evidence should be retrievable through trace IDs and span IDs across the lifecycle.

This enables:

- reconstruction of the execution path;
- verification of expected business events;
- isolation of failure causes;
- automated evidence collection;
- auditability of test decisions.

10 AI-Assisted Verification of Governed Controls

The paper does not reject generative AI. On the contrary, it argues for a more mature use of it.

AI can help verify controls defined by engineers.

Examples include:

- checking whether JSON payloads comply with schemas;
- verifying that generated datasets respect defined constraints;
- comparing observed traces against expected trace topology;
- detecting missing spans or missing audit events;
- checking whether expected artifacts were produced;
- identifying inconsistencies between requirements, test cases, and telemetry;
- generating scripts that perform these checks.

The difference is fundamental:

Weak pattern	**Stronger pattern**
Ask AI to generate tests and trust the output	Ask AI to help build validators for controls defined by engineers
Ask AI to invent test data	Ask AI to build auditable data-generation tools
Ask AI to define truth	Engineers define truth; AI explores variability around it
Ask AI to replace reasoning	Use AI to accelerate governed execution and verification

11 Generated Generators

One of the core contributions of the paper is the concept of generated generators.

Instead of asking generative AI to directly produce test data, engineers ask it to help build:

- scripts;
- data generators;

- lifecycle simulators;
- telemetry synthesizers;
- truth-matrix expanders;
- schema validators;
- trace validators.

These artifacts are governed as part of the test system.

11.1 Why Generators Matter

Generators are valuable because they are:

- inspectable;
- reviewable;
- versioned;
- reproducible;
- auditable;
- linked to requirements;
- linked to architecture decisions;
- controlled under configuration management.

If the system design changes, the impact can be traced across:

- input generation;
- expected outputs;
- expected signals;
- expected artifacts;
- telemetry expectations;
- validation scripts;
- evidence packages.

11.2 Generator as Test Evidence

The generator is not only a tool. It is part of the evidence chain.

A generated dataset without generator provenance is weak evidence. A generated dataset produced by a versioned generator, with parameters, seed, schema, and traceability to requirements, is stronger evidence.

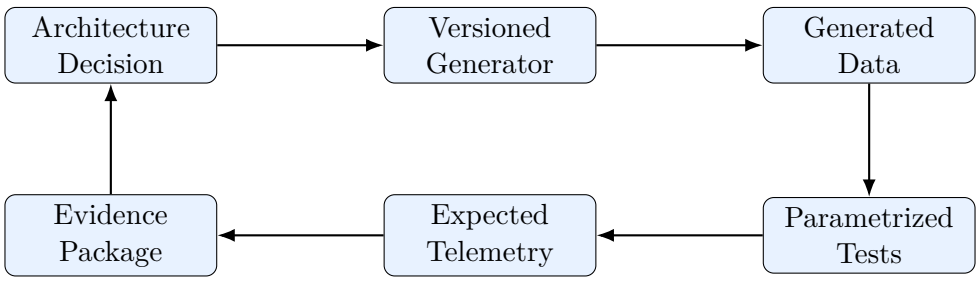

Figure 2: Generated generators as governed evidence infrastructure.

12 Practitioner Workflow

The model can be implemented as a workflow, and can be aligned with continuous delivery and CI/CD governance practices when evidence gates are treated as part of release readiness [21]. It does not require every organization to adopt a large methodology at once; it can be introduced progressively.

1. **Define the operational claim.** Example: "remote biometric verification rejects presentation attacks under defined conditions."
2. **Document assumptions.** Capture environment, architecture, data, instrumentation, policy, and dependency assumptions.
3. **Classify hypotheses versus assumptions.** Convert testable assumptions into falsifiable hypotheses where possible.
4. **Define the truth matrix.** Specify operational states, expected decisions, expected artifacts, and expected telemetry.
5. **Define evidence rules.** Specify what evidence is required for a pass, fail, inconclusive, or evidence-failure result.
6. **Build governed generators.** Use human design and AI assistance to produce auditable scripts and data generators.
7. **Verify preconditions.** Ensure that execution context is coherent before running tests.
8. **Execute parametrized scenarios.** Generate controlled variability and run tests.
9. **Validate outputs and telemetry.** Compare decisions, traces, logs, metrics, and artifacts against expectations.
10. **Produce evidence records.** Store machine-readable evidence packages.
11. **Update confidence and residual risk.** Interpret evidence against claims and assumptions.
12. **Decide.** Produce a go, conditional go, no-go, remediation, or risk-acceptance decision.

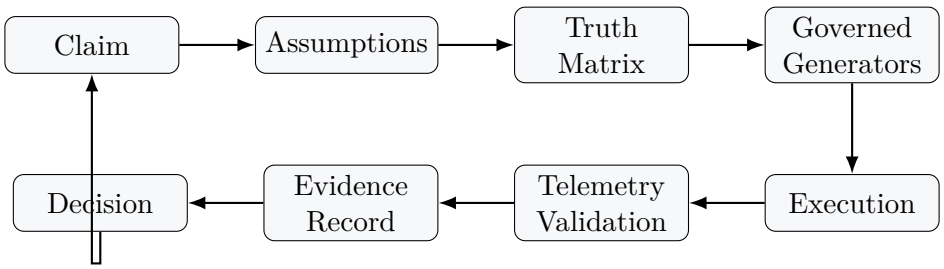

Figure 3: Implementation workflow for evidence-centric AI-assisted testing. The loop starts with a human-defined operational claim, moves through assumptions, truth matrices, governed generators, execution, telemetry validation, and evidence records, then returns to decision-making through confidence update and residual-risk review.

13 Preconditions, Inconclusive Results, and Defect Classification

A major failure mode in test governance is confusing three categories:

1. product defect;
2. invalid test context;
3. inconclusive evidence.

A robust test system must differentiate them.

13.1 Precondition Failure

If a required precondition is false, the test should not be interpreted as a product failure.

Examples:

- identity registry unavailable;
- biometric matcher not synchronized;
- certificate expired in the environment;
- test data corrupted;
- OpenTelemetry collector unavailable;
- feature flag not aligned with release package.

Such cases should trigger environment remediation or test-context correction.

13.2 Evidence Failure

If the expected output is correct but evidence is missing, the decision should be cautious. The system may have behaved correctly, but the assurance claim is not fully supported.

This is especially important in regulated systems where auditability is not optional.

14 Sociological Dimension: Generative AI and Premature Closure of Ambiguity

Testing in complex systems is not only a technical process. It is also a social process of constructing shared understanding.

Test design requires interaction among:

- architects;
- product experts;
- business analysts;
- QA engineers;
- operations teams;
- security experts;
- legal and compliance stakeholders;
- customer representatives;
- domain experts.

A test often reveals that the organization itself does not yet agree on what the system is supposed to do.

14.1 Why Generative AI Changes the Sociology of Testing

The socio-technical argument is not that AI is the first technology to affect testing work. The argument is that generative AI changes the form of organizational closure. Perrow's analysis of complex and tightly coupled systems warns that small interactions can produce consequences not visible from isolated components [16]. Latour's notion of black boxing explains how stabilized technical artifacts can hide the negotiations and assumptions that produced them [17]. Weick and

Sutcliffe's work on high-reliability organizations emphasizes preoccupation with failure, reluctance to simplify, sensitivity to operations, resilience, and deference to expertise [18]. These concepts matter because test artifacts are not neutral; they shape what an organization believes it knows.

Classic automation executes predefined instructions. Generative AI does something different: it produces fluent artifacts.

It can generate:

- test plans;
- acceptance criteria;
- risk analyses;
- requirements summaries;
- test scenarios;
- evidence reports;
- executive summaries.

This creates a specific sociological risk: premature closure of ambiguity.

> Generative AI may convert unresolved organizational ambiguity into polished apparent certainty.

A team may still not agree on:

- what the system should do;
- what evidence is sufficient;
- which assumption is fragile;
- which lifecycle state matters;
- what "valid identity verification" means operationally.

Yet the AI can produce a coherent-looking test plan.

This is more than a technical hallucination problem. It is an organizational knowledge problem. A well-formatted artifact may prematurely close a debate that should remain open until assumptions are clarified. In Latourian terms, unresolved reasoning can become black-boxed too early; in high-reliability terms, the organization may lose its reluctance to simplify [17, 18].

14.2 Quality of Assumptions

The quality of assumptions can be assessed through:

- explicit ownership;
- review date;
- evidence availability;
- falsifiability;
- dependency criticality;
- impact if false;
- conversion potential into hypotheses.

Weak assumptions are often hidden assumptions. The first governance benefit of this model is making them visible.

14.3 Preventing Governance Theater

Evidence-centric testing can be gamed if teams define weak assumptions to make results look better. Countermeasures include:

- independent assumption review;
- architecture peer review;
- traceability to operational incidents;
- challenge sessions with product and operations experts;
- explicit residual-risk ownership;
- evidence completeness metrics.

More concrete controls can be introduced progressively:

- **Assumption review boards:** lightweight peer reviews for assumptions, similar to code reviews, especially for release-critical claims.
- **Assumption red teaming:** assign a reviewer to falsify or stress-test the assumptions behind a truth matrix.
- **Incident-linked assumptions:** if an assumption was false in a past incident, outage, audit issue, or defect escape, it must be explicitly revalidated.
- **Automated weak-assumption detection:** flag tests that continue to pass even when critical assumptions are perturbed or removed.
- **Evidence-threshold gates:** prevent teams from reporting full pass when evidence completeness is below the agreed threshold.

The goal is to avoid turning evidence governance into compliance theater. The framework is valuable only if assumptions remain challengeable and evidence remains inspectable.

15 Novelty Relative to Existing Testing Traditions

This paper does not reject existing testing traditions. It builds on them.

15.1 Context-Driven Testing

Context-driven testing established that skilled human judgment, context, investigation, exploratory learning, and testing psychology are central to testing [14, 15, 13]. This paper strongly aligns with that tradition.

The novelty is not the claim that humans matter. The novelty is the extension of that principle into AI-assisted, observability-rich, system-of-systems environments.

Context-Driven Testing	This Paper Adds
Human judgment matters	Human-defined truth under AI-assisted generation
Context determines strategy	Operational context becomes a first-class validation object
Testing is investigation	Testing becomes evidence governance
Oracles matter	Telemetry becomes part of the oracle
Testers reason under uncertainty	Priors, assumptions, confidence updates, and residual risk are made explicit
Tools support testers	AI builds governed generators and validators rather than replacing test reasoning

15.2 Model-Based Testing

Model-based testing uses models to derive tests from expected system behavior [24]. Truth matrices and lifecycle-state reasoning resemble this tradition. The contribution here is not to replace model-based testing, but to connect it with:

- telemetry-as-oracle;
- generated generators;
- evidence records;
- confidence updates;
- operational-risk decisions.

15.3 Property-Based Testing

Property-based testing generates inputs to check invariant properties [23]. The generated-generator concept is compatible with this approach. The extension proposed here is to apply similar generative discipline to socio-technical operational claims, not only software properties.

15.4 Chaos Engineering

Chaos engineering introduces controlled disturbances to test resilience under failure conditions [22]. The present model aligns with its emphasis on operational assumptions and steady-state hypotheses. It extends the idea by requiring that the evidence chain itself be validated: not only did the system resist the disturbance, but did we produce the evidence required to trust that conclusion? This is consistent with safety engineering and high-reliability thinking, where awareness of weak signals and operational sensitivity matter as much as formal success [20, 18].

16 Case Studies

16.1 Case Study 1: Biometric Verification With Broken Audit Chain

Scenario. A remote biometric verification workflow rejects a presentation attack. The functional output is correct.

Traditional result. Test passed.

Evidence-centric result. Conditional failure.

Reason. The expected PAD alert was produced, but the audit-write span was missing and the trace chain could not reconstruct the end-to-end decision path.

Conclusion. The product behavior may be correct, but operational assurance is incomplete.

16.2 Case Study 2: Border-Control Workflow With Stale Watchlist

Scenario. A traveler is processed through a border-control identity verification workflow. The verification output is `APPROVE`.

Traditional result. Test passed if the expected output was approve.

Evidence-centric result. Inconclusive or invalid test context.

Reason. Precondition checks reveal that the watchlist synchronization timestamp is outside the acceptable freshness threshold.

Conclusion. The system decision cannot be interpreted without validating the operational context.

16.3 Case Study 3: Identity Lifecycle Drift Across Systems

Scenario. A credential is marked expired in the civil registry but remains active in a downstream verification service.

Traditional result. Individual service tests may pass locally.

Evidence-centric result. System-of-systems inconsistency.

Reason. The truth matrix expects lifecycle-state consistency across registry, policy engine, verification service, and audit system.

Conclusion. The defect is not localized assertion failure. It is an operational truth propagation failure.

17 AI-Assisted Doubt Under Human Governance

The paper should not be read as anti-AI. A more precise position is that AI can assist doubt, but should not own the final definition of truth.

Under human governance, AI can be used as:

- **a devil's advocate:** challenge a truth matrix by proposing edge cases, missing lifecycle states, or inconsistent assumptions;
- **an assumption gap detector:** scan requirements, architecture notes, test cases, and telemetry rules to identify assumptions that were never written down;
- **a scenario stressor:** propose degraded-mode, adversarial, latency, synchronization, or configuration-drift situations;
- **a trace reviewer:** compare expected trace topology against observed trace topology and flag missing spans or broken context propagation;
- **a consistency checker:** detect contradictions between expected outputs, lifecycle states, and evidence rules.

The governance boundary is essential. AI may propose doubts. Humans must decide which doubts matter, which assumptions become governed, which hypotheses require testing, and which residual risks can be accepted.

18 Tooling and Implementation Mapping

This whitepaper is intentionally tool-agnostic, but practical implementation can reuse existing platforms.

Need	Possible Tooling Direction
Truth matrix	CSV/YAML matrices, Cucumber scenario outlines, Gherkin tables, XRAY, Polarion, Jira, custom governance repositories
Parametrized generation	Python, property-based testing tools, Hypothesis, custom generators, model-based test generation
Generator governance	Git, pull requests, code review, semantic versioning, configuration management
Telemetry oracle	OpenTelemetry SDKs, collectors, Jaeger, Tempo, Prometheus, Loki, Elastic, custom span-topology validators
Evidence record	JSON evidence packages, Allure reports, XRAY evidence attachments, immutable artifact stores

Trace validation	Span-topology rules, trace ID continuity checks, expected event validation, schema-based telemetry verification
Risk dashboard	Grafana, custom BI dashboards, Jira/XRAY reports, STRAT-Q decision boards
Precondition verification	Shell/Python scripts, health checks, contract checks, environment drift detectors

The key point is not the tool itself. The key point is that test data, telemetry, evidence, and decisions must be connected through a governed chain.

18.1 Example End-to-End Tool Integration

A practical implementation could combine existing tools:

- define the truth matrix in YAML and expose selected scenarios as Cucumber/Gherkin scenario outlines;
- build parametrized generators in Python, optionally using property-based testing libraries for controlled variability;
- version generator code, truth matrices, and evidence rules in Git;
- instrument the system using OpenTelemetry SDKs and collect traces through an OpenTelemetry Collector;
- validate trace topology through a custom span validator that checks required spans, attributes, parent-child relationships, and trace continuity;
- store evidence packages in Allure, XRAY attachments, or an immutable artifact repository;
- enforce CI/CD gates in Jenkins, GitHub Actions, or equivalent tooling when evidence completeness falls below threshold.

This is deliberately incremental. Organizations do not need a new platform to begin. They need to connect existing quality, telemetry, and configuration-management practices into a single evidence chain.

19 Evaluation Metrics

The effectiveness of an evidence-centric AI-assisted testing approach should not be measured only by number of tests generated.

Recommended metrics include:

- percentage of tests linked to explicit claims;
- percentage of tests with documented assumptions;
- percentage of assumptions with owners and review dates;
- percentage of assumptions converted into falsifiable hypotheses;
- percentage of scenarios with truth-matrix coverage;
- precondition verification success rate;
- evidence completeness score;
- trace continuity rate;
- telemetry-oracle conformance rate;
- percentage of tests with retrievable logs, traces, metrics, and artifacts;

- number of architecture ambiguities discovered during test design;
- number of operational-context failures detected before execution;
- residual-risk reduction per test effort unit.

19.1 Evidence Completeness Score

An evidence completeness score can be defined as:

$$ECS = \frac{ObservedRequiredEvidence}{ExpectedRequiredEvidence}$$

where expected required evidence includes:

- output result;
- trace chain;
- required spans;
- required business events;
- logs;
- metrics;
- audit artifacts;
- generator provenance;
- environment precondition record.

For high-assurance systems, a passed assertion with low evidence completeness should not be considered a full pass.

20 Minimal Evidence Record

Each test execution should produce an evidence record similar to:

```
{
  "test_id": "BIO-VERIFY-ATTACK-042",
  "claim": "Presentation attacks are rejected under remote verification conditions",
  "assumptions": [
    {
      "id": "A-001",
      "statement": "PAD service is enabled",
      "owner": "Security QA",
      "review_date": "2026-06-01"
    },
    {
      "id": "A-002",
      "statement": "OTel collector is reachable",
      "owner": "DevOps",
      "review_date": "2026-06-01"
    }
  ],
  "prior": {
    "claim_valid_probability": 0.70,
    "prior_model": "Beta(7,3)",
    "source": ["architecture_review", "previous_incidents", "expert_elicitation"],
    "confidence_in_prior": "medium"
  },
  "input_generator": {
    "name": "biometric_variability_generator",
```

```
    "version": "1.4.2",
    "seed": "20260515-042",
    "repository_commit": "a17f92c"
  },
  "expected_decision": "REJECT",
  "expected_events": ["PAD_ALERT", "AUDIT_REJECTION"],
  "expected_trace_topology": [
    "capture",
    "quality_check",
    "pad_detection",
    "policy_decision",
    "audit_write"
  ],
  "actual_decision": "REJECT",
  "trace_id": "abc123",
  "evidence_status": "COMPLETE",
  "evidence_completeness_score": 1.0,
  "confidence_update": "positive",
  "residual_risk": "reduced but not eliminated",
  "decision": "conditional_go_pending_edge_case_review"
}
```


21 Counterarguments and Scope Boundaries

21.1 Objection 1: This Sounds Expensive

Yes. Evidence-centric governance has a cost.

The response is proportionality. Risk-management frameworks similarly emphasize tailoring governance to the context, impact, and use case [2, 3]. Not every system needs this level of rigor. However, in systems where identity, public trust, legal responsibility, safety, border operations, elections, or national-scale services are involved, the cost of insufficient evidence may exceed the cost of governance.

21.2 Objection 2: Not All Systems Need This

Correct.

This model should be scaled according to criticality:

- lightweight for low-risk digital services;
- moderate for business-critical platforms;
- rigorous for regulated identity, biometric, border, election, and mission-critical systems.

The purpose is not bureaucracy. The purpose is proportional assurance.

21.3 Objection 3: AI Can Help With Doubt

Yes, but with boundaries.

AI can surface edge cases, contradictions, missing scenarios, weak assumptions, and gaps in the truth matrix. However, humans remain accountable for defining evidence boundaries, accepting residual risk, and making operational decisions.

21.4 Objection 4: This Is Just Better Documentation

No.

Documentation records what was intended. Evidence-centric testing connects intention, execution, telemetry, generated data, and decision. It is not merely documentation; it is a governed evidence-production system.

22 Engineering Governance Model

We propose an engineering governance model with two planes:

- a **reasoning plane**, where humans define claims, assumptions, hypotheses, truth matrices, and evidence rules;
- an **industrialization plane**, where AI, scripts, and automation expand variability and verify governed controls.

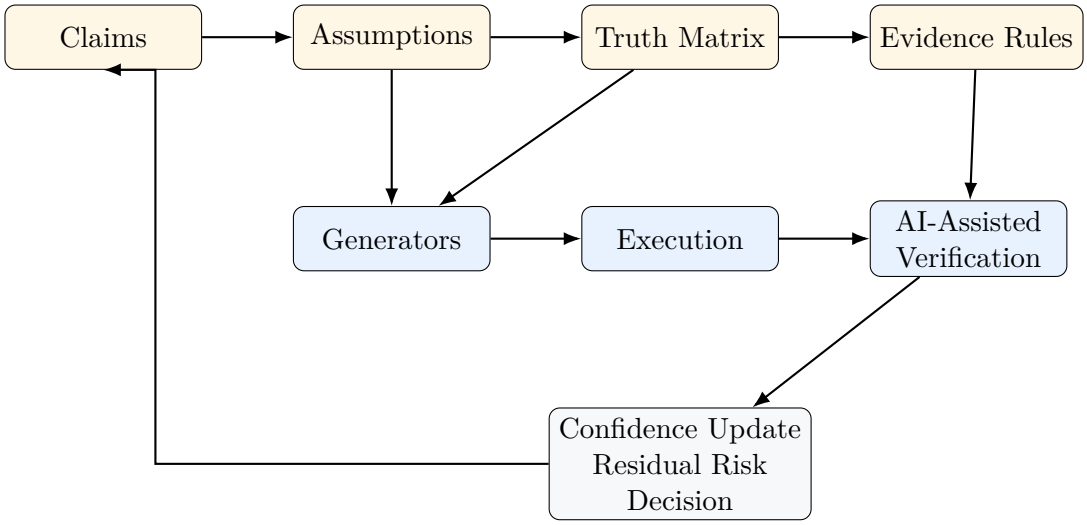

Figure 4: Human-defined reasoning plane and AI-assisted industrialization plane.

23 Implications for QA Leadership and Engineering Management

The proposed approach changes the role of QA leadership.

The QA leader is no longer only responsible for:

- test execution;
- defect detection;
- automation coverage;
- reporting.

The QA leader becomes responsible for:

- evidence governance;
- operational risk compression;
- traceability from requirement to evidence;
- quality of assumptions;
- confidence-aware decision support;
- cross-functional alignment around truth.

This is especially important in organizations where testing is often perceived as late-stage verification. In complex systems, testing should begin as early operational reasoning.

24 Limitations

This paper is a conceptual and architectural whitepaper. It does not claim that the proposed model is universally applicable without adaptation.

Limitations include:

- evidence-centric testing requires mature observability;
- truth matrices require domain expertise and governance effort;
- generated generators require configuration management discipline;
- telemetry validation may produce noise if instrumentation is unstable;
- sociological adoption may be harder than technical implementation;
- not all systems require this level of assurance.

However, for mission-critical identity, biometric, border, election, and public-service ecosystems, these costs may be justified by the operational and reputational risk profile.

25 Future Work

Future work should be prioritized according to feasibility, impact, and adoption cost.

Future Work Item	Feasibility	Impact	Priority	Rationale
CI/CD gates based on evidence completeness	High	High	P0	Directly actionable with existing pipelines and telemetry stores
Trace-topology validators for OpenTelemetry evidence	High	High	P0	Turns observability into executable evidence rules
Truth matrix integration with model-based testing and architecture descriptions [8, 24]	High	Medium	P1	Connects test design with system models and architecture rationale
Bayesian networks for residual-risk propagation	Medium	High	P1	Useful for critical systems where assumptions and claims are dependent

STRAT-Q-style residual-risk dashboards	Medium	Medium	P2	Helps executives see evidence, confidence, and residual risk together
Cross-functional truth reviews or evidence councils	Low/Medium	High	P2	High governance value but culturally harder to adopt
Application to PAD, identity lifecycle transitions, and border-control workflows	Medium	High	P1	Strong domain relevance for biometric and identity ecosystems

This roadmap intentionally starts with practical evidence gates and trace validation because they are easier to pilot and demonstrate quickly.

26 Conclusion

Generative AI can help produce tests. More importantly, it can help industrialize variability after engineers have defined the operational model.

But the strategic value of testing does not reside in mechanical production. It resides in doubt, priors, assumptions, hypotheses, architecture challenge, context validation, evidence design, and risk-informed decision-making.

AI can help build generators. AI can help verify governed controls. AI can help compare telemetry against expected patterns. AI can help scale parametrized exploration.

But engineers remain responsible for defining truth.

AI can generate plausibility.

Engineers remain accountable for operational truth.

The future of testing is not automated plausibility.

It is governed evidence.

References

[1] National Institute of Standards and Technology. *Digital Identity Guidelines, NIST Special Publication 800-63 Revision 4*. NIST, 2025. Available at: https://pages.nist.gov/800-63-4/

[2] National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST, 2023. Available at: https://www.nist.gov/itl/ai-risk-management-framework

[3] National Institute of Standards and Technology. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile, NIST AI 600-1*. NIST, 2024. DOI: https://doi.org/10.6028/NIST.AI.600-1.

[4] OpenTelemetry Project. *OpenTelemetry Documentation and Specification*. Cloud Native Computing Foundation. Available at: https://opentelemetry.io/docs/

[5] ISO/IEC. *ISO/IEC 30107-1:2016, Information technology - Biometric presentation attack detection - Part 1: Framework*. International Organization for Standardization, 2016. Available at: https://www.iso.org/standard/53227.html.

[6] ISO/IEC. *ISO/IEC 30107-3:2017, Information technology - Biometric presentation attack detection - Part 3: Testing and reporting*. International Organization for Standardization, 2017. Available at: https://www.iso.org/standard/67381.html.

[7] ISO/IEC/IEEE. *ISO/IEC/IEEE 29119 Software and systems engineering - Software testing*. International Organization for Standardization / International Electrotechnical Commission / IEEE. See for example: https://www.iso.org/standard/81291.html.

[8] ISO/IEC/IEEE. *ISO/IEC/IEEE 42010:2022, Software, systems and enterprise - Architecture description*. International Organization for Standardization / International Electrotechnical Commission / IEEE. Available at: https://www.iso.org/standard/74393.html.

[9] ETSI. *ETSI TS 119 461 v2.1.1 (February 2025), Electronic Signatures and Trust Infrastructures (ESI) - Policy and security requirements for trust service components providing identity proofing of trust service subjects*. European Telecommunications Standards Institute, 2025. Available at: https://www.etsi.org/deliver/etsi_ts/119400_119499/119461/02.01.01_60/ts_119461v020101p.pdf

[10] CNIL. *Biometrie: definitions et enjeux*. Commission Nationale de l'Informatique et des Libertes. Available at: https://www.cnil.fr/fr/biometrie

[11] National Institute of Standards and Technology. *Biometrics and Biometric Technology Evaluations*. NIST. Available at: https://www.nist.gov/programs-projects/biometrics

[12] Goodenough, J. B., and Gerhart, S. L. *Toward a theory of test data selection*. IEEE Transactions on Software Engineering, 1975. DOI: https://doi.org/10.1109/TSE.1975.6312836.

[13] Myers, G. J., Sandler, C., and Badgett, T. *The Art of Software Testing*. Wiley, third edition, 2011. Available at: https://www.wiley.com/en-us/The+Art+of+Software+Testing%2C+3rd+Edition-p-9781118031964.

[14] Kaner, C., Bach, J., and Pettichord, B. *Lessons Learned in Software Testing: A Context-Driven Approach*. Wiley, 2001. Available at: https://www.wiley.com/en-us/Lessons+Learned+in+Software+Testing%3A+A+Context+Driven+Approach-p-9780471081128.

[15] Bach, J. *Exploratory Testing Explained*. Satisfice, 2003. Available at: http://www.satisfice.com/articles/et-article.pdf.

[16] Perrow, C. *Normal Accidents: Living with High-Risk Technologies*. Princeton University Press, 1999. Available at: https://press.princeton.edu/books/paperback/9780691004129/normal-accidents.

[17] Latour, B. *Science in Action: How to Follow Scientists and Engineers Through Society*. Harvard University Press, 1987. Available at: https://www.hup.harvard.edu/books/9780674792913.

[18] Weick, K. E., and Sutcliffe, K. M. *Managing the Unexpected: Sustained Performance in a Complex World*. Jossey-Bass, 2015. Available at: https://www.wiley.com/en-us/Managing+the+Unexpected%3A+Sustained+Performance+in+a+Complex+World%2C+3rd+Edition-p-9781118862414.

[19] Gawande, A. *The Checklist Manifesto: How to Get Things Right*. Metropolitan Books, 2009. Available at: https://us.macmillan.com/books/9780312430009/thechecklistmanifesto.

[20] Hollnagel, E. *Safety-I and Safety-II: The Past and Future of Safety Management*. Ashgate, 2014. Bibliographic reference: https://portal.findresearcher.sdu.dk/en/publications/safety-i-and-safety-ii-the-past-and-future-of-safety-management.

[21] Humble, J., and Farley, D. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010. Available at: https://continuousdelivery.com/book/.

[22] Basiri, A., Behnam, N., de Rooij, R., Hochstein, L., Kosewski, L., Reynolds, J., and Rosenthal, C. *Chaos Engineering*. IEEE Software, 2016. DOI: https://doi.org/10.1109/MS.2016.60.

[23] Claessen, K., and Hughes, J. *QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs*. ACM SIGPLAN Notices, 2000. DOI: https://doi.org/10.1145/351240.351266.

[24] Utting, M., Pretschner, A., and Legeard, B. *A taxonomy of model-based testing approaches*. Software Testing, Verification and Reliability, 2012. DOI: https://doi.org/10.1002/stvr.456.

[25] Barrett, A. M., Hendrycks, D., Newman, J., and Nonnecke, B. *Actionable Guidance for High-Consequence AI Risk Management*. arXiv:2206.08966, 2022. Available at: https://arxiv.org/abs/2206.08966.

[26] Madhavan, K., Yazdinejad, A., Zarrinkalam, F., and Dehghantanha, A. *Quantifying Security Vulnerabilities: A Metric-Driven Security Analysis of Gaps in Current AI Standards*. arXiv:2502.08610, 2025. Available at: https://arxiv.org/abs/2502.08610.

A Appendix A: Example Generator Specification

```
generator:
  name: biometric_variability_generator
  version: 1.4.2
  seed_policy: deterministic_per_test_id

parameters:
  citizen_state:
    - new_applicant
    - active_traveler
    - watchlisted
    - expired_document
```

```
  biometric_context:
    - good_quality_capture
    - low_light
    - partial_face
    - presentation_attack

  operational_context:
    - normal
    - degraded_network
    - border_congestion
    - registry_latency

expected_artifacts:
  - generated_input_payload
  - expected_decision
  - expected_trace_topology
  - expected_audit_event
  - evidence_record
```

B Appendix B: Example Trace Validation Rule

```
rule: biometric_verification_trace_complete

required_spans:
  - capture
  - quality_check
  - biometric_match
  - policy_decision
  - audit_write

required_attributes:
  - transaction_id
  - identity_state
  - biometric_score
  - decision_reason

validation:
  same_trace_id: true
  missing_span_behavior: fail_evidence
  wrong_decision_behavior: fail_product
```

C Appendix C: Example Assumption Register

Assumption	Risk if False	Owner	Review Trigger
Matcher threshold is aligned with policy	False accept or false reject decision	Architecture / Product	Policy change
Presentation Attack Detection service is active	Spoofing scenario not valid	Security / QA	Release deployment
OpenTelemetry collector reachable	Evidence incomplete	DevOps / QA	Environment change
Test data representative	False confidence	QA / Product	Dataset update

Watchlist freshness within threshold	Wrong border decision	Operations / Security	Synchronization delay

D Appendix D: Acronyms

Acronym	**Meaning**
AI	Artificial Intelligence
CI/CD	Continuous Integration / Continuous Delivery or Deployment
ECS	Evidence Completeness Score
LLM	Large Language Model
MBT	Model-Based Testing
OTel	OpenTelemetry
PAD	Presentation Attack Detection
QA	Quality Assurance
STRAT-Q	Strategic Quality / risk-compression framing used as a conceptual foundation in this work